Meteorological Prediction of Hourly Ozone at Cabauw Using a Random Forest Model

Thursina Muhammad Ahsan
Student number: 8256675

Introduction

According to WHO (2021), surface ozone (O_3) is an important air pollutant that may damage lung function. It is formed through photochemical reactions involving precursor gases and solar radiation. This means, its concentration is strongly affected by meteorological condition (Otero et al., 2016). O_3 typically has a daily cycle following the exposure to the solar radiation, it increases during daytime and lowers at night. This study investigates to predict the daily cycle of O_3 and to know to what extent meteorological variables alone can explain hourly O_3 cycle at Cabauw-Wielsekade station in the Netherlands using a random forest model.

Data and methods

Hourly O_3 observations and meteorological data from Cabauw (NL10644) were used for 2016–2019. Cabauw location in a rural area is preferable to minimize local pollutant influences. Eight meteorological predictors were used: wind direction, wind speed, temperature, global radiation, hourly precipitation, pressure, cloud cover, and relative humidity.

The dataset is pre-processed to remove hours containing missing O_3 or meteorological data, which amounts to 3.0% of the data. The dataset also contains 608 negative O_3 measurements which were set to zero. The random forest model is configured with 30 trees and a minimum leaf size of five. It was trained on 25,450 hourly observations from 2016–2018 and then evaluated using 8,565 observations from 2019. The model performance was evaluated using correlation (r), R^2, root-mean-square error (RMSE), and mean bias. Additional experiments with varying number of trees (n = 10, 30, 100, 300) was done and also using each meteorological variable individually.

Results and discussion

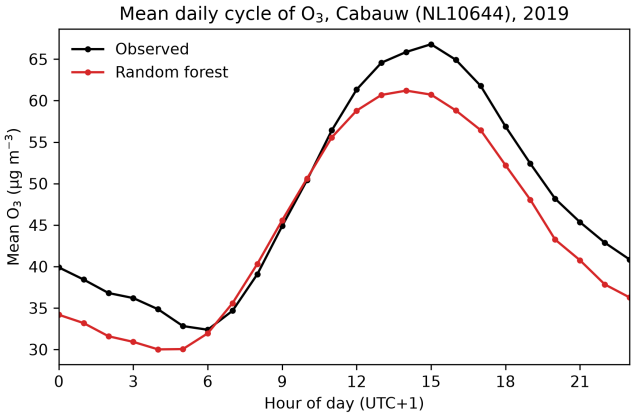


Figure 1: Mean daily cycle of O_3 at Cabauw during 2019. Black line shows observations and red line shows random-forest predictions.

The model reproduced the training data well, with $r = 0.96$, $R^2 = 0.91$, and an RMSE of $8.1\,\mu g\,m^{-3}$. Then it evaluated to predict 2019 data, which produced $r = 0.87$, $R^2 = 0.74$, and an RMSE of $15.5\,\mu g\,m^{-3}$. Mean observed O_3 in 2019 was $47.9\,\mu g\,m^{-3}$, compared with $44.5\,\mu g\,m^{-3}$ predicted by the model, so there is a slight bias of $-3.5\,\mu g\,m^{-3}$ (-7.3%). The statistics difference between training and prediction performance indicates that the model showed a decrease in predictive skill when applied outside the training period.

The model reproduced the overall daily O_3 cycle well (Fig. 1), although the amplitude was slightly underestimated. The daytime increase is consistent with stronger photochemical production under higher radiation and temperature, together with changes in boundary-layer mixing. The model also tended to underestimate O_3 during the night and late afternoon.

The scatter plot shows that between hourly observations and predictions (Fig. 2), the model captures most of the variability, but it struggles to predict the higher concentrations (Fan et al., 2022). The highest observed O_3

concentration in 2019 was 258 µg m^{-3}, whereas the maximum prediction was only 168 µg m^{-3}.

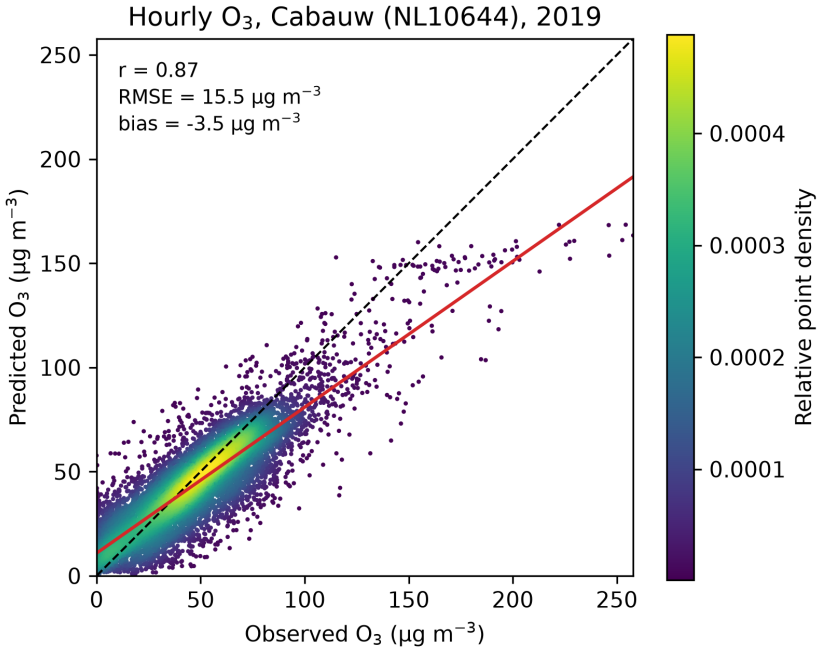


Figure 2: Hourly predicted versus observed O_3 at Cabauw in 2019. Colour indicates point density, the dashed line represents 1:1 agreement, and the red line is the least-squares regression.

Increasing the number of trees had only a small effect on model performance. The RMSE decreased from 15.80 µg m^{-3} with 10 trees to 15.54 µg m^{-3} with 30 trees and 15.42 µg m^{-3} with 100–300 trees, while the correlation remained approximately constant at $r = 0.87$.

The single-variable experiments show that relative humidity was the strongest individual predictor, with $R^2 = 0.47$, $r = 0.70$, and an RMSE of 22.1 µg m^{-3}, followed by temperature ($R^2 = 0.42$, $r = 0.67$) and global radiation ($R^2 = 0.23$, $r = 0.50$). The other meteorological parameters don't significantly explained the variability. In total, using all eight meteorological variables the model achieved $R^2 = 0.74$, which shows that there is no single meteorological variable that dominates the observed O_3 variability and there are other factors that influenced the concentration of O_3.

Conclusion

This experiment shows that meteorology explains a substantial part of the hourly O_3 variability at Cabauw. For the 2019 data, the random forest model predicts daily cycle reasonably well, achieved $r = 0.87$ and $R^2 = 0.74$. But it underestimated the mean concentration and the highest O_3 values. Relative humidity and temperature were the strongest individual predictors. Increasing the number of trees beyond n = 30 only showed minor improvements. Overall, the model captures the main meteorological control on O_3, but performs less well during extreme concentrations.

References

Fan, K., Dhammapala, R., Harrington, K., Lamastro, R., Lamb, B., and Lee, Y. (2022). Development of a machine learning approach for local-scale ozone forecasting: Application to Kennewick, WA. *Frontiers in Big Data*, 5, 781309.

Otero, N., Sillmann, J., Schnell, J. L., Rust, H. W., and Butler, T. (2016). Synoptic and meteorological drivers of extreme ozone concentrations over Europe. *Environmental Research Letters*, 11, 024005.

World Health Organization (2021). *WHO global air quality guidelines: particulate matter ($PM_{2.5}$ and PM_{10}), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide.* Geneva: World Health Organization.

Appendix: Python code

```
# Random forest model for O3 at Cabauw-Wielsekade (NL10644), steps 3-10.
# Train on 2016-2018 with 8 meteorological variables, predict 2019, evaluate,
# vary the number of trees (step 9) and use one variable at a time (step 10).
# Run: conda run -n college python rf_o3_cabauw_all.py
import os
import numpy as np
import pandas as pd
from scipy.stats import gaussian_kde
from sklearn.ensemble import RandomForestRegressor
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# ====================== settings ======================
STATION = "Rural_NL10644"
DATA_DIR = rf"D:\MAIO\{STATION}"
OUT_DIR = r"D:\MAIO\results"
TARGET = "o3"
FEATURES = ["wd", "ws", "t", "q", "hourly_rain", "p", "n", "rh"]
TRAIN_YEARS = [2016, 2017, 2018]
PRED_YEAR = 2019
NEGATIVE_TO_ZERO = True # True: set negative O3 to 0; False: keep them as measured
N_TREES = 30 # number of trees of the main model
N_TREES_LIST = [10, 30, 100, 300] # step 9
UNIT = "µg m$^{-3}$"

os.makedirs(OUT_DIR, exist_ok=True)
pd.set_option("display.width", 250)
pd.set_option("display.max_columns", 30)

# ====================== step 3: read data ======================
frames = []
for year in TRAIN_YEARS + [PRED_YEAR]:
    f = os.path.join(DATA_DIR, f"{STATION}_{year}.csv")
    frames.append(pd.read_csv(f, sep=";", na_values=["NA"], low_memory=False))
data = pd.concat(frames, ignore_index=True)
data["date"] = pd.to_datetime(data["date"])
data["year"] = data["date"].dt.year
data = data[["date", "year", TARGET] + FEATURES]

# ====================== step 5: missing data and negative values ======================
log = [f"Station: {STATION}, target: {TARGET}, features: {', '.join(FEATURES)}"]
log.append(f"Hours read: {len(data)}")
for year, g in data.groupby("year"):
    log.append(f"  {year}: {len(g)} hours, missing O3 {g[TARGET].isna().sum()}, "
               f"missing meteo {g[FEATURES].isna().any(axis=1).sum()}, "
               f"negative O3 {(g[TARGET] < 0).sum()}")

# Missing O3 is never filled (it is the value we predict and evaluate against).
# Missing wd means calm wind (no direction), so those hours are dropped too.
clean = data.dropna(subset=[TARGET] + FEATURES).copy()
log.append(f"Hours dropped (missing O3 or meteo): {len(data) - len(clean)} "
           f"({100 * (len(data) - len(clean)) / len(data):.1f}%)")

if NEGATIVE_TO_ZERO:
    n_neg = (clean[TARGET] < 0).sum()
    clean[TARGET] = clean[TARGET].clip(lower=0)
    log.append(f"Negative O3 set to 0: {n_neg} hours")

train = clean[clean["year"].isin(TRAIN_YEARS)]
pred = clean[clean["year"] == PRED_YEAR]
x_train, y_train = train[FEATURES].values, train[TARGET].values
x_pred, y_obs = pred[FEATURES].values, pred[TARGET].values
log.append(f"Training hours ({TRAIN_YEARS[0]}-{TRAIN_YEARS[-1]}): {len(train)}, "
           f"prediction hours ({PRED_YEAR}): {len(pred)}")

# ====================== step 4: build and train the random forest ======================
model = RandomForestRegressor(n_estimators=N_TREES, min_samples_leaf=5, random_state=42)
model.fit(x_train, y_train)
score = model.score(x_train, y_train)
log.append(f"R2 score (training data): {score:6.3f}")

# ====================== step 6: predict 2019 ======================
y_pred = model.predict(x_pred)
y_fit = model.predict(x_train)

out_train = train[["date", TARGET]].rename(columns={TARGET: "o3_obs"})
out_train["o3_pred"] = y_fit
out_pred = pred[["date", TARGET]].rename(columns={TARGET: "o3_obs"})
out_pred["o3_pred"] = y_pred
out_train.to_csv(os.path.join(OUT_DIR, "o3_cabauw_train_2016-2018.csv"), index=False, float_format="%.2f")
out_pred.to_csv(os.path.join(OUT_DIR, "o3_cabauw_pred_2019.csv"), index=False, float_format="%.2f")

importance = pd.Series(model.feature_importances_, index=FEATURES).sort_values(ascending=False)
log.append("Feature importance (impurity-based, training):")
for name, value in importance.items():
    log.append(f"  {name:12s} {value:.3f}")
with open(os.path.join(OUT_DIR, "run_log.txt"), "w") as f:
    f.write("\n".join(log) + "\n")
print("\n".join(log))
```

```
# ======================== step 8: statistics ========================
# Hourly values, and daily means (days with at least 18 valid hours).
rows = []
for period, df in [("2016-2018␣(training)", out_train), ("2019␣(prediction)", out_pred)]:
    daily = df.set_index("date").resample("D").agg(["mean", "count"])
    daily = daily[daily[("o3_obs", "count")] >= 18]
    for resolution, obs, mod in [("hourly", df["o3_obs"].values, df["o3_pred"].values),
                                 ("daily␣mean", daily[("o3_obs", "mean")].values,
                                  daily[("o3_pred", "mean")].values)]:
        diff = mod - obs
        r = np.corrcoef(obs, mod)[0, 1]
        rows.append({
            "period": period,
            "resolution": resolution,
            "n": len(obs),
            "mean␣obs": obs.mean(),
            "mean␣pred": mod.mean(),
            "bias␣(pred-obs)": diff.mean(),
            "NMB␣(%)": 100 * diff.sum() / obs.sum(),
            "r": r,
            "r2": r ** 2,
            "R2␣(1-SSres/SStot)": 1 - (diff ** 2).sum() / ((obs - obs.mean()) ** 2).sum(),
            "RMSE": np.sqrt((diff ** 2).mean()),
            "MAE": np.abs(diff).mean(),
            "std␣obs": obs.std(),
            "std␣pred": mod.std(),
            "max␣obs": obs.max(),
            "max␣pred": mod.max(),
        })
stats = pd.DataFrame(rows)
stats.to_csv(os.path.join(OUT_DIR, "statistics_step8.csv"), index=False, float_format="%.3f")
print("\nStep␣8␣statistics:")
print(stats.round(2).to_string(index=False))

# ======================== step 7: plots ========================
plt.rcParams.update({"font.size": 10})

# Figure A: mean daily cycle in 2019 (mean of all hourly values per hour of day)
out_pred["hour"] = out_pred["date"].dt.hour
cycle = out_pred.groupby("hour")[["o3_obs", "o3_pred"]].mean()
cycle_out = cycle.rename(columns={"o3_obs": "o3_obs_mean", "o3_pred": "o3_pred_mean"})
cycle_out.insert(0, "n_hours", out_pred.groupby("hour")["o3_obs"].count())
cycle_out.to_csv(os.path.join(OUT_DIR, "fig_daily_cycle_2019_values.csv"), float_format="%.2f")

fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(cycle.index, cycle["o3_obs"], color="black", lw=1.5, marker="o", ms=3, label="Observed")
ax.plot(cycle.index, cycle["o3_pred"], color="tab:red", lw=1.5, marker="o", ms=3, label="Random␣forest")
ax.set_xlim(0, 23)
ax.set_xticks(range(0, 24, 3))
ax.set_xlabel("Hour␣of␣day␣(UTC+1)")
ax.set_ylabel(f"Mean␣O$_3$␣({UNIT})")
ax.set_title("Mean␣daily␣cycle␣of␣O$_3$,␣Cabauw␣(NL10644),␣2019")
ax.legend(loc="upper␣left", frameon=False)
fig.tight_layout()
fig.savefig(os.path.join(OUT_DIR, "fig_daily_cycle_2019.png"), dpi=300)
plt.close(fig)

# Figure B: density scatter of hourly predicted vs observed O3 in 2019
obs = out_pred["o3_obs"].values
mod = out_pred["o3_pred"].values
density = gaussian_kde(np.vstack([obs, mod]))(np.vstack([obs, mod]))
order = density.argsort() # plot the densest points last (on top)
slope, intercept = np.polyfit(obs, mod, 1)
row = stats[(stats["period"] == "2019␣(prediction)") & (stats["resolution"] == "hourly")].iloc[0]
lim = max(obs.max(), mod.max()) # axis limit = maximum data value

fig, ax = plt.subplots(figsize=(5.5, 4.8))
sc = ax.scatter(obs[order], mod[order], c=density[order], s=4, cmap="viridis", edgecolors="none")
ax.plot([0, lim], [0, lim], color="black", lw=1, ls="--")
ax.plot([0, lim], [intercept, intercept + slope * lim], color="tab:red", lw=1.5)
ax.set_xlim(0, lim)
ax.set_ylim(0, lim)
ax.set_aspect("equal")
ax.set_xlabel(f"Observed␣O$_3$␣({UNIT})")
ax.set_ylabel(f"Predicted␣O$_3$␣({UNIT})")
ax.set_title("Hourly␣O$_3$,␣Cabauw␣(NL10644),␣2019")
ax.text(0.04, 0.96, f"r␣=␣{row['r']:.2f}\nRMSE␣=␣{row['RMSE']:.1f}␣{UNIT}\nbias␣=␣{row['bias␣(pred-obs)']:.1f}␣{UNIT}",
        transform=ax.transAxes, va="top", fontsize=9)
cb = fig.colorbar(sc, ax=ax, shrink=0.85)
cb.set_label("Relative␣point␣density")
fig.tight_layout()
fig.savefig(os.path.join(OUT_DIR, "fig_scatter_2019.png"), dpi=300)
plt.close(fig)

# Figure C: full 2019 hourly series (for checking, too dense for the abstract)
fig, ax = plt.subplots(figsize=(10, 3.5))
ax.plot(out_pred["date"], out_pred["o3_obs"], color="black", lw=0.4, label="Observed")
ax.plot(out_pred["date"], out_pred["o3_pred"], color="tab:red", lw=0.4, alpha=0.8, label="Random␣forest")
ax.set_ylabel(f"Hourly␣O$_3$␣({UNIT})")
ax.set_title("Cabauw␣(NL10644),␣2019,␣hourly")
ax.legend(loc="upper␣left", frameon=False)
```

```
fig.tight_layout()
fig.savefig(os.path.join(OUT_DIR, "fig_hourly_2019_full.png"), dpi=200)
plt.close(fig)

# ====================== step 9: number of trees ======================
rows = []
for n_trees in N_TREES_LIST:
    model9 = RandomForestRegressor(n_estimators=n_trees, min_samples_leaf=5, random_state=42)
    model9.fit(x_train, y_train)
    for period, obs, mod in [("2016-2018␣(training)", y_train, model9.predict(x_train)),
                             ("2019␣(prediction)", y_obs, model9.predict(x_pred))]:
        diff = mod - obs
        r = np.corrcoef(obs, mod)[0, 1]
        rows.append({
            "n_estimators": n_trees,
            "period": period,
            "n": len(obs),
            "mean␣obs": obs.mean(),
            "mean␣pred": mod.mean(),
            "bias␣(pred-obs)": diff.mean(),
            "NMB␣(%)": 100 * diff.sum() / obs.sum(),
            "r": r,
            "r2": r ** 2,
            "R2␣(1-SSres/SStot)": 1 - (diff ** 2).sum() / ((obs - obs.mean()) ** 2).sum(),
            "RMSE": np.sqrt((diff ** 2).mean()),
            "MAE": np.abs(diff).mean(),
            "std␣obs": obs.std(),
            "std␣pred": mod.std(),
            "max␣obs": obs.max(),
            "max␣pred": mod.max(),
        })
table9 = pd.DataFrame(rows)
table9.to_csv(os.path.join(OUT_DIR, "step9_n_estimators.csv"), index=False, float_format="%.3f")
print("\nStep␣9␣(number␣of␣trees):")
print(table9.round(2).to_string(index=False))

# ====================== step 10: one meteorological variable at a time ======================
# All runs use the same hours (rows with all 8 variables present), so only the variable differs.
runs = [("all␣8", FEATURES)]
for var in FEATURES:
    runs.append((var, [var]))

rows = []
for name, variables in runs:
    model10 = RandomForestRegressor(n_estimators=N_TREES, min_samples_leaf=5, random_state=42)
    model10.fit(train[variables].values, y_train)
    for period, obs, mod in [("2016-2018␣(training)", y_train, model10.predict(train[variables].values)),
                             ("2019␣(prediction)", y_obs, model10.predict(pred[variables].values))]:
        diff = mod - obs
        r = np.corrcoef(obs, mod)[0, 1]
        rows.append({
            "variables": name,
            "period": period,
            "n": len(obs),
            "mean␣obs": obs.mean(),
            "mean␣pred": mod.mean(),
            "bias␣(pred-obs)": diff.mean(),
            "NMB␣(%)": 100 * diff.sum() / obs.sum(),
            "r": r,
            "r2": r ** 2,
            "R2␣(1-SSres/SStot)": 1 - (diff ** 2).sum() / ((obs - obs.mean()) ** 2).sum(),
            "RMSE": np.sqrt((diff ** 2).mean()),
            "MAE": np.abs(diff).mean(),
            "std␣obs": obs.std(),
            "std␣pred": mod.std(),
            "max␣obs": obs.max(),
            "max␣pred": mod.max(),
        })
table10 = pd.DataFrame(rows)
table10.to_csv(os.path.join(OUT_DIR, "step10_single_variables.csv"), index=False, float_format="%.3f")
print("\nStep␣10␣(single␣variables):")
print(table10.round(2).to_string(index=False))

print(f"\nAll␣results␣written␣to␣{OUT_DIR}")
```